

```
serializers.deserialize("json", data, ignorenonexistent=True)
```

3.21 Formatların Serileştirilmesi

Django, bazıları üçüncü taraf Python modülleri yüklemenizi gerektiren bir dizi serileştirme formatını destekler. Modüller;

Identi- fier	Information
xml	Serializes to and from a simple XML dialect.
json	Serializes to and from JSON.
jsonl	Serializes to and from JSONL.
yaml	Serializes to YAML (YAML Ain't a Markup Language). This serializer is only available if PyYAML is installed.

Xml

Temel Xml Değiştirme formatı şu şekilde gözükmektedir.

```
<?xml version="1.0" encoding="utf-8"?>
<django-objects version="1.0">
  <object pk="123" model="sessions.session">
    <field type="DateTimeField" name="expire_date">2013-01-16T08:16:59.844560+00:00</
    ↪field>
    <!-- ... -->
  </object>
</django-objects>
```

Serileştirilen veya serileştirilmeden çıkarılan nesne koleksiyonunun tamamı, içersinde birden fazla <object> ögesi barındıran bir <django-object> etiketi ile temsil edilir. Bu nesnelerin her biri “pk” ve “model” kısmı, uygulama adı (session) ve modelin küçük harflerle yazılmış adının adı (session) bir nokta ile birleştirilmesiyle oluşur. Ögenin her bir alanı “type” ve “name” özelliklerine sahip bir <field> etiketi şeklinde dönüştürülür.Ögenin metin içeriği , saklanması gereken değeri temsil eder.

“Foreign

Keys ” ve diğer alakalı alanlar birazdaha farklı ele alınır.

```
<!-- ... -->
<field to="contenttypes.contenttype" name="content_type" rel="ManyToOneRel">9</field>
```

(diğer sayfadan devam eder)

```
<!-- ... -->
```

```
</object>
```

Bu örnekte, Pk 27 adında bir izin tanımı (auth.Permission) Pk 9 adında ise içerik türüne (contenttypes.ContentType) bağlıdır. ManyToMany(Çoka Çok) bu bağlantıyı kendi üzerine bağlayan model aracılığıyla dışa aktarılır. Örneğin, (auth.User) modelinin (auth.Permission) modeli ile böyle bir ilişkisi vardır.

```
<!-- ... -->
```

```
<field to="auth.permission" name="user_permissions" rel="ManyToManyRel">
```

```
<object pk="46"></object>
```

```
<object pk="47"></object>
```

```
</field>
```

```
</object>
```

Bu örnek, var olan kullanıcıyı Pk 46 ve Pk 47 izin modelleri ile ilişkilendirir.

Kontrol Karakterleri

Serileştirilecek içerik, Xml, 1.0 standardında kabul edilmeyen kontrol karakterleri içeriyorsa, serileştirme bir ValueError istinası ile başarısız olur. Ayrıca W3C'nin HTML, XHTML, XML ve Kontrol Kodları açıklamasını okuyun.

Json

Önceki örnek verilerle devam edildiğinde, json formatında aşağıdaki şekilde dönüştürülecektir.

```
[
  {
    "pk": "4b678b301dfd8a4e0dad910de3ae245b",
    "model": "sessions.session",
    "fields": {
      "expire_date": "2013-01-16T08:16:59.844Z",
      # ...
    },
  },
]
```

Buradaki biçimlendirme XML'e göre biraz daha basittir. Tüm koleksiyon sadece bir dizi olarak temsil edilir;

ve nesneler üç özelliğe sahip JSON nesneleriyle temsil edilir: "pk", "model" ve "fields". "fields" ise

Django dokümantasyonu, versiyon

6.0.5.dev20260430142210

Her “Field” ın

adını ve değerini sırasıyla özellik ve özellik-değer olarak içeren bir nesnedir.

Yabancı anahtarlar (Foreign keys), bağlantılı nesnenin birincil anahtarını (PK) özellik değeri olarak alır. Çoktan çoğa ilişkiler (ManyToMany-relations), onları tanımlayan model için serileştirilir ve bir birincil anahtar (PK) listesi olarak temsil edilir.

Tüm Django çıktıların değiştirilmeden **json**'a aktarılamayacağını unutmayın. Örneğin, serileştirilecek bir nesne içinde özel (custom) bir türünüz varsa, bunun için özel bir **json** kodlayıcı (encoder) yazmanız gerekecektir. Şuna benzer bir şey iş görecektir:"

```
from django.core.serializers.json import DjangoJSONEncoder

class LazyEncoder(DjangoJSONEncoder):
    def default(self, obj):
        if isinstance(obj, YourCustomType):
            return str(obj)
        return super().default(obj)
```

Ardından, serializers.serialize() fonksiyonuna cls=LazyEncoder parametresini geçirebilirsiniz:

```
from django.core.serializers import serialize

serialize("json", SomeModel.objects.all(), cls=LazyEncoder)
```

Ayrıca GeoDjango'nun özelleştirilmiş bir GeoJSON serileştiricisi (serializer) sunduğunu da unutmayın.

DjangoJSONEncoder

```
class django.core.serializers.json.DjangoJSONEncoder
```

JSON serileştirici (serializer), kodlama işlemi için DjangoJSONEncoder sınıfını kullanır. JSONEncoder sınıfının bir alt sınıfı olan bu yapı, şu ek veri tiplerini de işleyebilir:

datetime

ECMA-262'de tanımlandığı şekliyle YYYY-AA-GGTSS:dd:ss.sssZ veya YYYY-AA-GGTSS:dd:ss.sss+SS:DD biçiminde bir dizgi (string).

date

ECMA-262'de tanımlandığı şekliyle YYYY-AA-GG biçiminde bir dizgi.

time

ECMA-262'de tanımlandığı şekliyle SS:DD:ss.sss biçiminde bir dizgi.

timedelta

ISO-8601'de tanımlandığı şekliyle bir süreyi temsil eden dizgi. Örneğin; timedelta(days=1, hours=2, seconds=3.4) ifadesi 'P1DT02H00M03.400000S' olarak temsil edilir.

Django dokümantasyonu, versiyon

6.0.5.dev20260430142210

Decimal, Promise (django.utils.functional.lazy() nesneleri), UUID: Nesnenin dizgi (string) gösterimi.

JSONL

JSONL, 'JSON Lines' (JSON Satırları) anlamına gelir. Bu formatta nesneler yeni satırlarla birbirinden ayrılır ve her satır geçerli bir JSON nesnesi içerir. JSONL ile serileştirilmiş veriler şuna benzer:

```
{"pk": "4b678b301dfd8a4e0dad910de3ae245b", "model": "sessions.session", "fields": {...}}
{"pk": "88bea72c02274f3c9bf1cb2bb8cee4fc", "model": "sessions.session", "fields": {...}}
{"pk": "9cf0e26691b64147a67e2a9f06ad7a53", "model": "sessions.session", "fields": {...}}
```

JSONL can be useful for populating large databases, since the data can be processed line by line, rather than being loaded into memory all at once.

Yaml

YAML serileştirmesi JSON'a oldukça benzer. Nesne listesi, "pk", "model" ve "fields" anahtarlarıyla bir dizi eşleme olarak serileştirilir. Her alan yine bir eşlemedir; anahtar alanın adı, değer ise değerdir.

```
- model: sessions.session
  pk: 4b678b301dfd8a4e0dad910de3ae245b
  fields:
    expire_date: 2013-01-16 08:16:59.844560+00:00
```

Referans alanları yine birincil anahtar (PK) veya birincil anahtarlar dizisi ile temsil edilir.

Özel serileştirme biçimleri

Varsayılan formatlara ek olarak, özel bir serileştirme formatı oluşturabilirsiniz. Örneğin, bir CSV serileştirici (serializer) ve serisizleştirici (deserializer) ele alalım. İlk olarak, bir Serializer ve Deserializer sınıfı tanımlayın. Bu sınıflar, mevcut serileştirme formatı sınıflarının üzerine yazabilir (onları geçersiz kılabilir):

Liste38: path/to/custom_csv_serializer.py

```
import csv

from django.apps import apps
from django.core import serializers
from django.core.serializers.base import DeserializationError

class Serializer(serializers.python.Serializer):
```

(sonraki sayfada devam eder)

(önceki sayfanın devamı)

```
def get_dump_object(self, obj):
    dumped_object = super().get_dump_object(obj)
    row = [dumped_object["model"], str(dumped_object["pk"])]
    row += [str(value) for value in dumped_object["fields"].values()]
    return ",".join(row), dumped_object["model"]

def end_object(self, obj):
    dumped_object_str, model = self.get_dump_object(obj)
    if self.first:
        fields = [field.name for field in apps.get_model(model)._meta.fields]
        header = ",".join(fields)
        self.stream.write(f"model,{header}\n")
        self.stream.write(f"{dumped_object_str}\n")

def getvalue(self):
    return super(serializers.python.Serializer, self).getvalue()

class Deserializer(serializers.python.Deserializer):
    def __init__(self, stream_or_string, **options):
        if isinstance(stream_or_string, bytes):
            stream_or_string = stream_or_string.decode()
        if isinstance(stream_or_string, str):
            stream_or_string = stream_or_string.splitlines()
        try:
            objects = csv.DictReader(stream_or_string)
        except Exception as exc:
            raise DeserializationError() from exc
        super().__init__(objects, **options)

    def _handle_object(self, obj):
        try:
            model_fields = apps.get_model(obj["model"])._meta.fields
            obj["fields"] = {
                field.name: obj[field.name]
                for field in model_fields
                if field.name in obj
            }
            yield from super()._handle_object(obj)
        except (GeneratorExit, DeserializationError):
```

(sonraki sayfada devam eder)

3.21 Django objelerini serileştirmek

(Önceki sayfanın devamı)

```
        raise
    except Exception as exc:
        raise DeserializationError(f"Error deserializing object: {exc}") from exc
```

Ardından, serileştirici tanımlarını içeren modülü `SERIALIZATION_MODULES` ayarınıza ekleyin

```
SERIALIZATION_MODULES = {
    "csv": "path.to.custom_csv_serializer",
    "json": "django.core.serializers.json",
}
```

Sağlanan serileştirme formatlarının her birine bir Deserializer (serisizleştirici) sınıf tanımı eklendi.

3.21.4 Doğal Anahtarlar(Natural Keys)

Yabancı anahtarlar (foreign keys) ve çoktan çoğa (many-to-many) ilişkiler için varsayılan serileştirme stratejisi, ilişkideki nesnelerin birincil anahtar (primary key) değerlerini serileştirmektir. Bu strateji çoğu nesne için iyi çalışır, ancak bazı durumlarda zorluklara neden olabilir

ContentType modeline referans veren bir yabancı anahtara sahip nesneler listesini ele alalım. Eğer bir içerik türüne (content type) atıfta bulunan bir nesneyi serileştirecekseniz, her şeyden önce o içerik türüne atıfta bulunmanın bir yoluna ihtiyacınız vardır. ContentType nesneleri veritabanı senkronizasyon işlemi sırasında Django tarafından otomatik olarak oluşturulduğundan, belirli bir içerik türünün birincil anahtarını tahmin etmek kolay değildir; bu değer, migrate komutunun nasıl ve ne zaman çalıştırıldığına bağlı olarak değişecektir. Bu durum, başta Permission (İzin), Group (Grup) ve User (Kullanıcı) olmak üzere, nesneleri otomatik olarak oluşturan tüm modeller için geçerlidir.

Uyarı

Otomatik olarak oluşturulan nesneleri asla bir fixture (hazır veri dosyası) veya diğer serileştirilmiş verilere dahil etmemelisiniz. Ans eseri, fixture içindeki birincil anahtarlar (primary keys) veritabanındakiyle eşleşebilir ve bu durumda fixture'ın yüklenmesi hiçbir etki yaratmaz. Çok daha muhtemel olan eşleşmeme durumunda ise, fixture yükleme işlemi bir `IntegrityError` (Bütünlük Hatası) ile başarısız olacaktır.

Bir de kolaylık meselesi vardır. Bir nesneye atıfta bulunmak için tam sayı türündeki bir ID (kimlik numarası) her zaman en kullanışlı yol değildir; bazen daha doğal bir referans faydalı olabilir

İşte bu nedenlerden dolayı Django, **doğal anahtarları** (**natural keys**) sunar. Doğal anahtar, birincil anahtar (primary key) değerini kullanmadan, bir nesne örneğini benzersiz şekilde tanımlamak için kullanılabilen değerler demetidir (tuple)

Deserialization of natural keys

Aşağıdaki iki modeli ele alalım:

```
from django.db import models

class Person(models.Model):
    first_name = models.CharField(max_length=100)
    last_name = models.CharField(max_length=100)

    birthdate = models.DateField()

    class Meta:
        constraints = [
            models.UniqueConstraint(
                fields=["first_name", "last_name"],
                name="unique_first_last_name",
            ),
        ]

class Book(models.Model):
    name = models.CharField(max_length=100)
    author = models.ForeignKey(Person, on_delete=models.CASCADE)
```

Normal şartlarda, Book (Kitap) için serileştirilmiş veriler, yazara atıfta bulunmak için bir tam sayı (integer) kullanır. Örneğin, JSON formatında bir Book şu şekilde serileştirilebilir.

...

(Sonraki sayfadan devam eder)

-----3.21 Django objelerini serileştirmek

(önceki sayfanın devamı)

```
{"pk": 1, "model": "store.book", "fields": {"name": "Mostly Harmless", "author": 42}}
...
```

"Bu, bir yazara atıfta bulunmak için pek de doğal bir yol değildir. Yazara ait birincil anahtar (primary key) değerini bilmenizi gerektirir; ayrıca bu birincil anahtar değerinin sabit ve öngörülebilir olmasını zorunlu kılar. Ancak, Person (Kişi) modeline doğal anahtar (natural key) yönetimini eklersek, fixture (hazır veri dosyası) çok daha insancıl bir hale gelir. Doğal anahtar yönetimini eklemek için, Person modeli için `get_by_natural_key()` metoduna sahip varsayılan bir Manager (Yönetici) tanımlarsınız. Bir Person örneğinde, iyi bir doğal anahtar, ad ve soyad çiftinden oluşabilir:

```
from django.db import models

class PersonManager(models.Manager):
    def get_by_natural_key(self, first_name, last_name):
        return self.get(first_name=first_name, last_name=last_name)

class Person(models.Model):
    first_name = models.CharField(max_length=100)
    last_name = models.CharField(max_length=100)
    birthdate = models.DateField()

    objects = PersonManager()

    class Meta:
        constraints = [
            models.UniqueConstraint(
                fields=["first_name", "last_name"],
                name="unique_first_last_name",
            ),
        ]
```

Artık kitaplar, Person (Kişi) nesnelere atıfta bulunmak için bu doğal anahtarı kullanabilir:

```
{
    "pk": 1,
    "model": "store.book",
    "fields": {"name": "Mostly Harmless", "author": ["Douglas", "Adams"]},
}
```

(önceki sayfanın devamı)

...

Bu serileştirilmiş veriyi yüklemeye çalıştığınızda Django; ["Douglas", "Adams"] ifadesini gerçek bir Person (Kişi) nesnesinin birincil anahtarına (primary key) dönüştürmek için `get_by_natural_key()` metodunu kullanacaktır.

Not

Doğal anahtar (natural key) için kullandığınız alanlar ne olursa olsun, bir nesneyi benzersiz (unique) bir şekilde tanımlayabilmelidir. Bu durum genellikle, modelinizin doğal anahtarındaki alan veya alanlar için bir benzersizlik artına (tek bir alan üzerinde `unique=True`, ya da birden fazla alan üzerinde bir `UniqueConstraint` veya `unique_together`) sahip olacağı anlamına gelir.

Ancak, benzersizliğin veritabanı düzeyinde zorunlu tutulması şart değildir. Eğer bir dizi alanın fiilen benzersiz olacağından emininseniz, bu alanları yine de doğal anahtar olarak kullanabilirsiniz.

Birincil anahtar (primary key) olmayan nesnelerin serisizleştirilmesi (deserialization) sırasında, her zaman model yöneticisinin (manager) bir `get_by_natural_key()` metoduna sahip olup olmadığı kontrol edilir; eğer varsa, serisizleştirilen nesnenin birincil anahtarını doldurmak için bu metod kullanılır.

Doğal

Anahtarların Serileştirilmesi

Peki, bir nesneyi serileştirirken Django'nun doğal bir anahtar (natural key) döndürmesini nasıl sağlarsınız? İlk olarak, başka bir metod eklemeniz gerekir – bu sefer metodun kendisini modele eklemelisiniz:

```
class Person(models.Model):
    first_name = models.CharField(max_length=100)
    last_name = models.CharField(max_length=100)
    birthdate = models.DateField()

    objects = PersonManager()

    class Meta:
        constraints = [
            models.UniqueConstraint(
                fields=["first_name", "last_name"],
                name="unique_first_last_name",
            ),
        ]

    def natural_key(self):
        return (self.first_name, self.last_name)
```

Bu metod her zaman bir doğal anahtar demeti (tuple) döndürmelidir – bu örnekte, (ad, soyad). Ardından, `serializers.serialize()` fonksiyonunu çağırdığınızda, `use_natural_foreign_keys=True` veya `use_natural_primary_keys=True` argümanlarını sağlarsınız:

```
>>> serializers.serialize(
...     "json",
...     [book1, book2],
...     indent=2,
...     use_natural_foreign_keys=True,
...     use_natural_primary_keys=True,
... )
```

use_natural_foreign_keys=True belirtildiğinde Django, bu metodu (natural_key) tanımlayan türdeki nesnelere yapılan tüm yabancı anahtar (foreign key) atıflarını serileştirmek için natural_key() metodunu kullanacaktır. use_natural_primary_keys=True belirtildiğinde ise Django, serisizleştirme (deserialization) sırasında hesaplanabileceği için bu nesnenin serileştirilmiş verilerinde birincil anahtarı (primary key) sağlamayacaktır:

```
...
{
  "model": "store.person",
  "fields": {
    "first_name": "Douglas",
    "last_name": "Adams",
    "birth_date": "1952-03-11",
  },
}
...
```

Bu durum, serileştirilmiş verileri mevcut bir veritabanına yüklemeniz gerektiğinde ve serileştirilmiş birincil anahtar (primary key) değerinin zaten kullanımda olmadığını garanti edemediğinizde ve serisizleştirilen (deserialized) nesnelerin aynı birincil anahtarları korumasını sağlamanız gerekmediğinde kullanışlı olabilir.

Serileştirilmiş veri oluşturmak için `dumpdata` kullanıyorsanız, doğal anahtarlar üretmek için `dumpdata --natural-foreign` ve `dumpdata --natural-primary` komut satırı bayraklarını (flags) kullanın.

Not

Hem `natural_key()` hem de `get_by_natural_key()` metodlarını tanımlamanıza gerek yoktur. Eğer Django'nun serileştirme (serialization) sırasında doğal anahtarlar çıktı vermesini istemiyor, ancak doğal anahtarları yükleme (load) yeteneğini korumak istiyorsanız, `natural_key()` metodunu uygulamamayı seçebilirsiniz.

Aksine, eğer (garip bir nedenden dolayı) Django'nun serileştirme sırasında doğal anahtarlar çıktı vermesini istiyor ancak bu anahtar değerlerini yükleyememesini istiyorsanız, sadece `get_by_natural_key()` metodunu tanımlamayın.

Doğal anahtarlar ve ileriye yönelik referanslar

Bazen doğal yabancı anahtarlar (natural foreign keys) kullandığınızda, bir nesnenin henüz serisizleştirilmemiş (deserialized) başka bir nesneye atıfta bulunan bir yabancı anahtara sahip olduğu verileri serisizleştirmeniz gerekecektir. Buna **'ileriye yönelik referans' (forward reference)** denir. Örneğin, fixture dosyanızda aşağıdaki nesnelerin bulunduğunu varsayalım:

```
...
{
    "model": "store.book",
    "fields": {"name": "Mostly Harmless", "author": ["Douglas", "Adams"]},
},
...
{"model": "store.person", "fields": {"first_name": "Douglas", "last_name": "Adams"}},
...
```

Bu durumu yönetmek için, `serializers.deserialize()` fonksiyonuna `handle_forward_references=True` parametresini geçmeniz gerekir. Bu işlem, `DeserializedObject` örnekleri üzerinde `deferred_fields` (ertelenmiş alanlar) özneliğini ayarlayacaktır. Bu özneliğin `None` olmadığı `DeserializedObject` örneklerini takip etmeniz ve daha sonra bu örnekler üzerinde `save_deferred_fields()` metodunu çağırmanız gerekecektir.

Tipik kullanım şu şekildedir:

```
objs_with_deferred_fields = []

for obj in serializers.deserialize("json", data, handle_forward_references=True):
    obj.save()
    if obj.deferred_fields is not None:
        objs_with_deferred_fields.append(obj)

for obj in objs_with_deferred_fields:
    obj.save_deferred_fields()
```

Bunun çalışabilmesi için, referans veren model üzerindeki `ForeignKey` (yabancı anahtar) alanı `null=True` özelliğine sahip olmalıdır.

"Bir fixture (hazır veri dosyası) içindeki nesnelerin sıralamasına dikkat ederek, ileriye yönelik referansları (forward references) açıkça yönetmek zorunda kalmaktan kaçınmak genellikle mümkündür. Buna yardımcı olmak için, `dumpdata --natural-foreign` seçeneğini kullanan `dumpdata` çağrıları, `natural_key()` metoduna sahip herhangi bir modeli, standart birincil anahtar (primary key) nesnelerini serileştirmeden önce serileştirecektir. Ancak, bu her zaman yeterli olmayabilir. Eğer doğal anahtarınız başka bir nesneye atıfta bulunuyorsa (doğal anahtarın bir parçası olarak başka bir nesneye yönelik bir yabancı anahtar veya doğal anahtar kullanarak), o zaman doğal anahtarın bağımlı olduğu nesnelerin, doğal anahtar onlara ihtiyaç duymadan önce serileştirilmiş verilerde yer aldığından emin olmanız gerekir. Bu sıralamayı kontrol etmek için, `natural_key()` metodlarınız üzerinde bağımlılıklar (**dependencies**) tanımlayabilirsiniz. Bunu, `natural_key()` metodunun kendisine bir `dependencies` özneliği (attribute) atayarak yaparsınız.

Örneğin, yukarıdaki örnekteki Book (Kitap) modeline bir doğal anahtar ekleyelim:

```
class Book(models.Model):
    name = models.CharField(max_length=100)
    author = models.ForeignKey(Person, on_delete=models.CASCADE)

    def natural_key(self):
        return (self.name,) + self.author.natural_key()
```

Bir Book (Kitap) için doğal anahtar, kitabın ismi ve yazarının birleşimidir. Bu durum, Person (Kişi) modelinin Book modelinden önce serileştirilmesi gerektiği anlamına gelir. Bu bağımlılığı tanımlamak için fazladan bir satır ekleriz:

```
def natural_key(self):
    return (self.name,) + self.author.natural_key()

natural_key.dependencies = ["example_app.person"]
```

Bu tanım, tüm Person (Kişi) nesnelerinin herhangi bir Book (Kitap) nesnesinden önce serileştirilmesini sağlar. Buna bağlı olarak, Book modeline atıfta bulunan herhangi bir nesne de, hem Person hem de Book nesneleri serileştirildikten sonra serileştirilecektir.

Test Ve Açık Uçlu Sorular

1) Django'da bir nesneyi serileştirirken, birincil anahtar (primary key) yerine daha okunabilir ve anlamlı bir referans (örneğin ad ve soyad kombinasyonu) kullanılmasına ne ad verilir?

- A) Serileştirme Modeli
- B) Natural Keys (Doğal Anahtarlar)
- C) Foreign Keys(Yabancı Anahtar)
- D)Json Kodlayıcı

1) Dokümanda belirtildiği üzere, JSONL (JSON Lines) formatının büyük veritabanlarını doldururken standart JSON formatına göre sağladığı en önemli avantaj nedir?

Cvp) JSONL formatında veriler tümüyle belleğe yüklenmek yerine satır satır işlenebildiği için büyük veritabanlarında bellek (RAM) kullanımı açısından çok daha verimlidir.